ShipmentIQ Technical Design Document

GitHub URL: https://github.com/Manjunath-G-K/ShipmentIQ

Live application URL: Not verified; deployment remains a manual release step.

1. Purpose and final scope

ShipmentIQ is a customs-broker documentation workbench. It stores a shipment and private source documents, extracts proposed structured values, exposes confidence and page/source evidence, compares current values across documents, supports human review and correction history, reports review outcomes, and returns a backend-gated provisional CSV.

It does not file on a customs portal, classify HSN codes, calculate duty, provide importer tracking, certify regulatory compliance, or decide which document is correct. Extraction is proposed data; confidence is not verification.

2. Architecture and technology choices

The implementation is a modular monolith:

```
React + TypeScript + Vite
        · credentialed REST + CSRF
FastAPI application
  auth/RBAC · shipments · documents/storage
  OCR/extraction · review/validation · exports/reporting
        .                          .
PostgreSQL + Alembic       private local storage adapter
```

- Backend: FastAPI, SQLAlchemy 2, Pydantic v2, Alembic, PostgreSQL.
- Frontend: React 18, TypeScript, Vite, plain scoped CSS; no UI framework or state-management dependency.
- Authentication: short-lived JWT in an HttpOnly cookie, signed double-submit CSRF cookie/header, server-side role reload from PostgreSQL.
- Storage: private generated-name local storage behind a DocumentStorage protocol. Cloud object storage is intentionally not part of the initial deployment.
- Processing: synchronous extraction to keep status/failure visible without adding Redis, Kafka, a worker, or another service.

Migrations 0001 through 0006 create the foundation, authentication, shipments, documents, extraction persistence, and review/validation/revision tables. Part 3 CSV export is streamed and does not add an export-attempt table.

3. Data model

- users, roles, user_roles: active identities and database-backed roles.
- shipments: unique reference, importer, supplier, origin, description, creator, timestamps, and backend-controlled status.
- shipment_documents: one invoice, packing list, or bill of lading per type, private storage key, checksum, content type, and owner.
- document_extractions: latest provider names, status, page count, JSON result, timestamps, and safe failure code.
- review_fields: flattened field path, original extracted value, current human-reviewed value, value type, confidence, source references, status, and optimistic version.
- field_revisions: append-only actor/timestamp/action/value history.
- shipment_validations: one current result per deterministic rule, evidence JSON, evaluator, timestamp, and STALE state.

4. Main workflows

Authentication and authorization

Login validates credentials and sets JWT/CSRF cookies. Every protected request reloads the active user and roles from PostgreSQL. Object scope is enforced for shipments, documents, fields, revisions, validations, reports, and exports. State-changing requests require an allowed Origin and matching CSRF cookie/header. Executive-only mutation paths and manager-only reports are enforced on the backend.

Document intake and access

Uploads require a declared supported type and are validated by extension, content signature, size, and basic structure. Storage uses opaque generated names and private permissions. Downloads re-check shipment scope and document ownership relationship before opening bytes. The browser viewer consumes the authorized download endpoint; it never receives a public storage path.

OCR and extraction

OCRProvider and AIExtractionProvider are replaceable protocols. Local OCR uses embedded PDF text when available and Tesseract/PyMuPDF/Pillow for scanned PDF pages and images. The default AI path is a free labelled-text parser that only accepts explicitly labelled values. An OpenAI-compatible provider is opt-in through environment configuration. Provider results are normalized narrowly and validated by strict Pydantic models; malformed responses become safe persisted failures. Missing or unreadable values remain missing/needs review.

Human review

The review screen keeps an authorized original document beside field values. Each field displays current value, extracted value, confidence, flags, and page/locator references. Executives can correct values; executives and managers can confirm or reject according to the current role policy. Corrections require an expected version and create an append-only revision. Rejected values are cleared rather than replaced with a guess.

Validation and stale state

The deterministic rules are invoice total versus line-item total, package count across available documents, gross weight across available documents, and net weight across available documents. Missing comparison evidence is UNRESOLVED, never PASSED. Numeric comparisons use exact normalized values because no client tolerance was supplied. Any correction or rejection locks and marks existing validation rows STALE; validation must be explicitly rerun.

CSV export

The backend recomputes eligibility for every request. It requires a readable completed extraction, required invoice values and line items, no rejected/low-confidence/unconfirmed populated fields, and all validation rules in PASSED state. The export service locks the shipment row while evaluating eligibility and taking the CSV snapshot, so concurrent corrections serialize before or after a complete decision. An export response marks the shipment EXPORTED; a later correction changes the shipment to action-required and invalidates validation results.

The layout is explicitly provisional because the client's actual Excel import template was not supplied. Columns are: shipment_reference, importer, supplier, origin_country, invoice_number, invoice_date, currency, incoterm, invoice_total, item_description, quantity, unit_price, hsn_code, packages, gross_weight_kg, net_weight_kg, containers.

5. UI and design decisions

The client includes sign-in, shipment listing, shipment creation, shipment detail/review, document viewer/upload, validation/export controls, revision history, and manager accuracy reporting. It uses white/beige paper surfaces, dark green navigation, restrained rust accents, responsive grid behavior, and explicit loading/empty/error states. Buttons call real APIs; the frontend never decides authorization or export eligibility.

6. Testing strategy

- Backend unit tests cover configuration, security, storage validation, extraction schemas/providers, review rules, and export layout/required fields.
- PostgreSQL integration tests cover migrations, authentication/RBAC, object access, upload races, extraction persistence, review/revisions, stale validations, blocked/cleared export, and concurrent expected-version updates.
- Frontend tests cover credentialed API parsing and backend error propagation.
- The Vite production build is run as a TypeScript and bundling check.
- A browser-level Playwright suite is not included. Desktop/tablet/mobile browser smoke checks and production hosting verification remain manual release steps.

7. Decisions and assumptions

- Assumption: 0.80 is the default low-confidence threshold because the brief supplies no threshold; it is configurable.
- Assumption: exact numeric equality is used because the brief supplies no rounding/tolerance policy.
- Assumption: the local labelled-text provider is the no-cost default; paid AI is never required unless explicitly configured.
- Assumption: the current role policy permits manager confirmation/rejection and validation, while executive-only corrections/uploads/CSV generation remain server-enforced.
- The demo seeder creates only clearly marked synthetic documents and is disabled in production.

8. Limitations and security findings

- The actual filing template, manager approval policy, retention policy, supplier identity policy, and legal/regulatory acceptance criteria remain unresolved.
- No accuracy, performance, cost, 95%-field-accuracy, or amended-entry reduction target is claimed; the repository has no labelled 200-shipment evaluation set.
- Local storage is for development/demo only. Render's filesystem is ephemeral, so uploaded documents can disappear after restart or redeployment in the initial deployment. HTTPS, secret injection, PostgreSQL backups, and an approved provider/data-use policy are still required for a production service.
- Login rate limiting, refresh-token rotation, password recovery, and full malware scanning are not implemented.
- The npm audit review found six development-tool vulnerabilities in the Vitest 2 dependency tree after safe same-major updates to Vite 6.4.5 and Vitest 2.1.9. Eliminating the remaining advisories requires a major Vitest upgrade; that was not forced because it could change the test tool contract. Production dependencies report zero vulnerabilities with npm audit --omit=dev.

9. Submission readiness

The local workflow, migrations, synthetic demo data, backend integration suite, frontend tests, production frontend build, deployment runbook, and this TDD are present. Submission remains conditional on a successful Pages/Render/Neon configuration and smoke test, a separately supplied demo credential set, and the client's real CSV template if filing integration is required.